

Task 1 Report

Hex, Memory and Stack Calculator Report

Group Members:

[Abdi Raqiib Hassan, Abubakar Mohammed Jibrilla, Abdulfatah Mohamed, Mohammed Ahmed,
Muhammad Zeeshan, Zohaib Hussain]

Overview

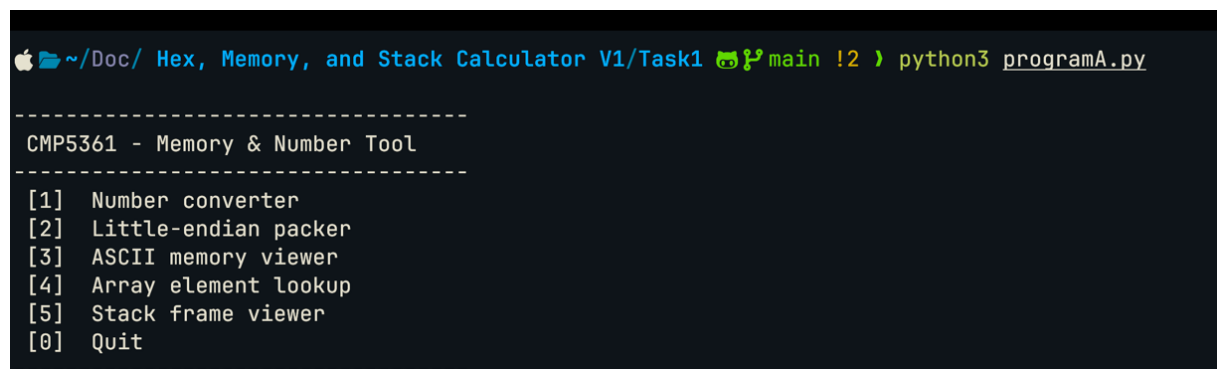
This is a Python application which is a menu-driven command-line tool that we build as a group. It illustrates our combined understanding of the fundamental ideas in low level memory and computer architecture. In this report, we will cover topics including hexadecimal notation, binary representation, array addressing along with stack frames. Simulating how a real computer would store, read and interpret values in real time memory.

We used a python dictionary with the variable name “toy_memory” to visualise how a toy memory model with each value being a single byte and each key being a memory location. We ensured that the menu interface was structured accordingly and kept tidy for user friendliness and the logic is simple to verify with unit tests because the essential functionality is divided into small testable functions

Menu Option Structure

Option	Name	Description
0	Exit system	It exists the user from the application
1	Number converter	It converts decimal to hex and 16-bit binary, signed vs unsigned.
2	Little-Endian Packer	It splits a 16-bit number into two bytes
3	Ascii memory Viewer	It stores strings as bytes in memory with a null terminator
4	Array Element Lookup	Address calculation using base and index size, read/write option
5	Stack Frame Viewer	It's a simplified bp offset model and register styled output

Screenshot: Main Menu Option (0 – 5)



```

~ / Doc / Hex, Memory, and Stack Calculator V1 / Task1 main !2 ) python3 programA.py

-----
CMP5361 - Memory & Number Tool
-----
[1] Number converter
[2] Little-endian packer
[3] ASCII memory viewer
[4] Array element lookup
[5] Stack frame viewer
[0] Quit
-----
```

Figure 1: main menu programming running on terminal

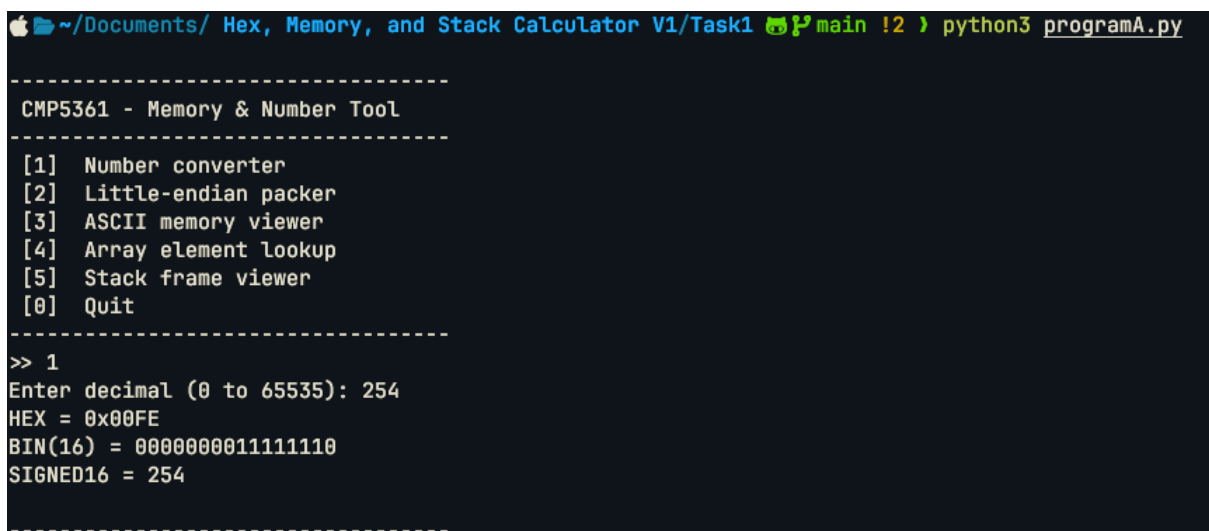
Option Details & Screenshots

Option 1 – Number Converter

When a decimal integer is entered between the numbers 0 and 65535 by the user. The software that we constructed will then transform into a signed two complement interpretation, a 16-bit binary string that will always be precisely 16 characters wide and uppercase such as (0x00FF). in more detail numbers which are less than 32768 will be positive and numbers which are greater will be 32768 binding to negative values. And example of that scenario would be 65535 will become -1.

Input	Output	Concept
A decimal number from 0 - 65535	HEX = {HEX_VALUE} BIN (16) = {BIN16_VALUE} SIGNED16={SIGNED16}	There will be number bases and two complement signed integers

Screenshot: Option 1 – Number Converter



```
~/Documents/ Hex, Memory, and Stack Calculator V1/Task1 main !2 › python3 programA.py
-----
CMP5361 - Memory & Number Tool
-----
[1] Number converter
[2] Little-endian packer
[3] ASCII memory viewer
[4] Array element lookup
[5] Stack frame viewer
[0] Quit
-----
>> 1
Enter decimal (0 to 65535): 254
HEX = 0x00FE
BIN(16) = 0000000011111110
SIGNED16 = 254
-----
```

Figure 2:converting a number

Option 2 – Little Endian Packer

The next option in the menu is the little-endian packer. When the user enters a 16-bit number and a memory address which can be accepted in a decimal or hex format, the software will then divide into its high byte which is the next eight bits and for the low byte it's the least significant eight bits. It will then write them to successive memory address in little endian sequence and then reads them back to verify proper storage. Furthermore, to demonstrate that the procedure is reversible it will then use the two bytes to recreate the original number.

Input	Output	Concept
16-bit value and memory address	LOW BYTE = NUMBER HIGH BYTE = NUMBER UNPACKED = NUMBER MEM [] = 0X00 READ MEM [] = 0X00	Little-endian, ordering of bytes and memory mapped storage

Screenshot: Option 2 – Little-endian packer

```
~/Doc/ Hex, Memory, and Stack Calculator V1/Task1 main !1 » python3 programA.py

-----
CMP5361 - Memory & Number Tool
-----

[1] Number converter
[2] Little-endian packer
[3] ASCII memory viewer
[4] Array element lookup
[5] Stack frame viewer
[0] Quit
-----

>> 2
Enter a 16-bit number: 1000
Target address (hex like 0x2000 or decimal): 0x2000
LOW BYTE = 232
HIGH BYTE = 3
UNPACKED = 1000
MEM[0x2000] = 0xE8
MEM[0x2001] = 0x03
READ MEM[0x2000] = 0xE8
READ MEM[0x2001] = 0x03
```

Figure 3: memory write and read-back output

Option 3 – ASCII Memory Viewer

The next option is to input a string of a maximum of 10 characters by the user. The program will mimic how C-style strings are kept in memory by storing each character as its ASCII byte value, beginning at its base address 0x1000 and then appending a null terminator byte 0x00 at the subsequent location. The overall length of the string is then printed after each location and its byte value in a memory dump.

Input	Output	Concept
A string of max 10 characters	Address of the byte value per character, null terminator and length line (how long the character is)	Ascii encoding, memory layout and null terminated strings.

Screenshot: option 3 – ASCII memory viewer

```
🍏 ~/Doc/ Hex, Memory, and Stack Calculator V1/Task1 🐍 main !1 > python3 programA.py

-----
CMP5361 - Memory & Number Tool
-----
[1] Number converter
[2] Little-endian packer
[3] ASCII memory viewer
[4] Array element lookup
[5] Stack frame viewer
[0] Quit
-----
>> 3
Type a string (10 chars max): BORING
0x1000 : 0x42
0x1001 : 0x4F
0x1002 : 0x52
0x1003 : 0x49
0x1004 : 0x4E
0x1005 : 0x47
0x1006 : 0x00
LENGTH (until 0x00) = 6
-----
```

Figure 4: ascii memory dump for a string (e.g. BORING)

Option 4 – Array Element Lookup

The fourth option in the menu is the array element lookup program. The user will be prompted to provide the element index, one or two base addresses, and the element size in bytes when they are given the option to choose the 4th option (array element lookup). The user then is presented with a choice between two being read or write mode only after the address has been determined. Then, after that process, the value that is entered will be placed into the selected toy memory at the given address while it is in write mode. This process shows the hardware level access and the management of arrays.

Input	Output	Concept
Base address, Element index, bytes per element, operation(read/write)	Address line, value to store, write or read confirmation	Array indexing, memory access and pointer logic.

Screenshot option 4 – array element lookup

```
~/Documents/ Hex, Memory, and Stack Calculator V1/Task1 main !2 ?1 } python3 programA.py

-----
CMP5361 - Memory & Number Tool
-----
[1] Number converter
[2] Little-endian packer
[3] ASCII memory viewer
[4] Array element lookup
[5] Stack frame viewer
[0] Quit
-----
>> 4
Base address: 2004
Element index: 2
Bytes per element (1 or 2): 2
Operation - read or write: write
ADDRESS = 2004 + 2*2 = 0x07D8
Value to store: 500
WRITE size=2 value=500 to ADDRESS 0x07D8
-----
```

Figure 5: array write then read at calculated address

Option 5 – Stack Frame Viewer

The last option in the menu is the stack frame viewer, whereby two integer numbers representing the parameters of the function are entered by the user. After that process, a simple diagram is produced showing where these parameters would be on the stack in relation to the base pointer. This will then be shown by the return address at bp. The first parameter, as seen in the screenshot below, is bp+2, and the second parameter will be bp+4. After that, a second diagram is produced showing registers with the first parameter in ax and the second in bx, and the addition in ax (ax + bx) = first and second input of numbers, for example, 10 and 20, the ax output will be 30.

Input	Output	Concept
Two integer numbers	Bp offset table , ax/bx register and ax + bx result	CPU registers, base pointer offsets, call stack

Screenshot option 5 – stack frame viewer

```
~/Documents/ Hex, Memory, and Stack Calculator V1/Task1 main !2 ?1 » python3 programA.py

-----
CMP5361 - Memory & Number Tool
-----
[1] Number converter
[2] Little-endian packer
[3] ASCII memory viewer
[4] Array element lookup
[5] Stack frame viewer
[0] Quit
-----
>> 5
First param (a): 10
Second param (b): 20
bp      : RETURN
bp+2    : a = 10
bp+4    : b = 20
AX = 10
BX = 20
AX (AX+BX) = 30
-----
```

Figure 6: Stack frame and register output

Testing: Test_A.py

The test file in the project is called testA.py. The file has 10 meaningful unit tests covering the 8 required tests and 2 more. As stated in the brief, it is difficult to test everything inside input () and print (). To make it much simpler, we put the main calculations into small functions. We wrote the tests using pytest out of the two options. We imported the core functions from programA.py. There are no menus or inputs, keeping things simple and only showing the success of all 10 tests.

How to run the tests

Pytest testA.py

What each test checks

Test	What it checks	Spec requirement
test_pack_unpack_zero	pack16(0) - both bytes 0, unpack returns 0	boundary: 0
test_pack_unpack_one	pack16(1) - low=1 high=0, unpack returns 1	boundary: 1
test_pack_unpack_255	pack16(255) - low=255 high=0, unpack returns 255	boundary: 255
test_pack_unpack_256	pack16(256) - low=0 high=1 unpack returns 256	boundary: 256
test_pack_unpack_65535	pack16(65535) - both bytes 255, unpack returns 65535	boundary: 65535
test_ascii_dump_simple	ascii_dump('HELLO') —first byte 0x48, last byte 0x00	ascii dump
test_array_address	array_address(1000, 3, 2) = 1006	Array addressing
test_stack_frame	show_stack(10, 20) — RETURN, a=10, b=20 in correct rows	Stack frame
test_binary_length	BIN(16) is 16 characters for 0, 1, 255, 256, 65535	bin(16) always 16 bits
test_signed_and_memory	SIGNED16: 65535→-1, 32768→-32768. Memory write 1000, read back 1000	SIGNED16 + memory write/read

Screenshot: Successful tests output

```
~/Documents/ Hex, Memory, and Stack Calculator V1/Task1  main [2 ?1 ] pytest  20:33:06
===== test session starts =====
platform darwin -- Python 3.12.6, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/abdiraqibhassan/Documents/ Hex, Memory, and Stack Calculator V1/Task1
collected 10 items

test_A.py ..... [100%]

===== 10 passed in 0.01s =====
```

Figure 7: all 10 tests successfully passing in the terminal

References

Get Started - pytest documentation (no date).

<https://docs.pytest.org/en/stable/getting-started.html>

W3Schools.com (no date).

https://www.w3schools.com/python/numpy/numpy_array_indexing.asp.

Jones, L. (2024) *Pointers in Python: What's the point?* <https://realpython.com/pointers-in-python/>.

GeeksforGeeks (2025) *Memory Management in Python*.

<https://www.geeksforgeeks.org/python/memory-management-in-python/>.